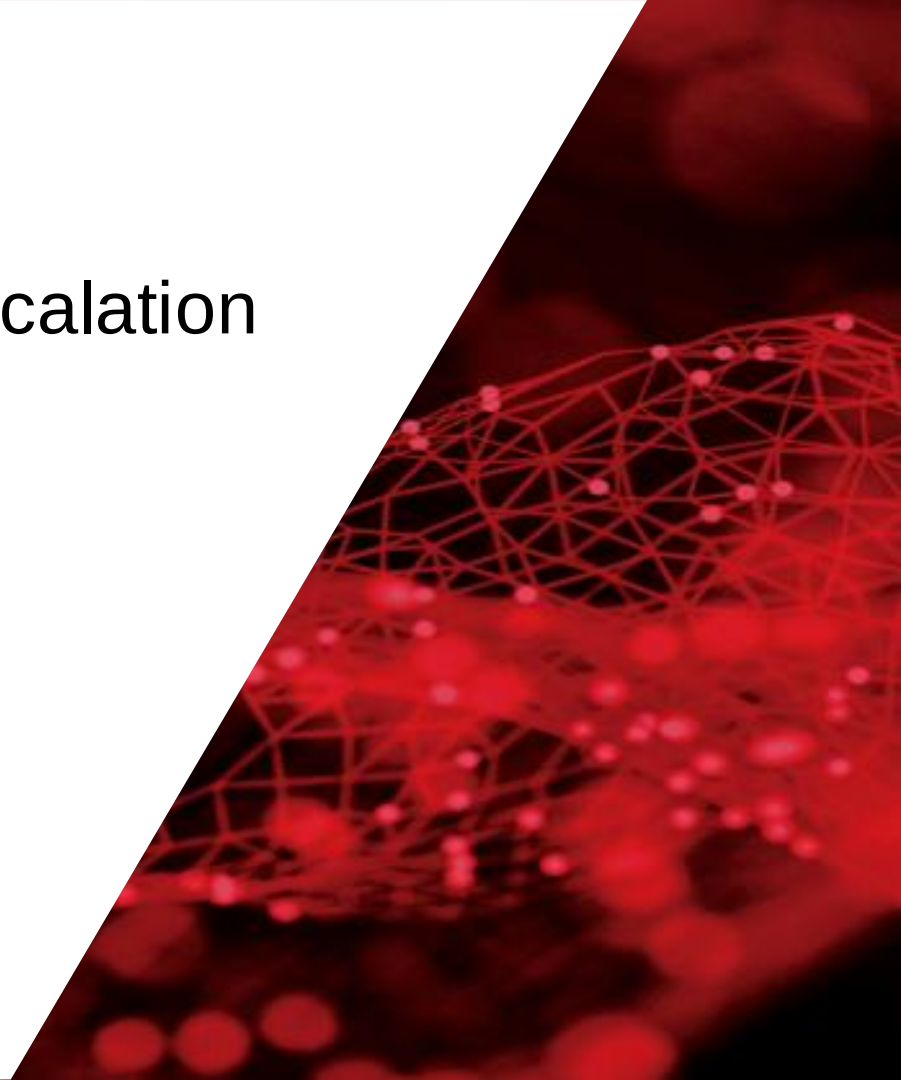
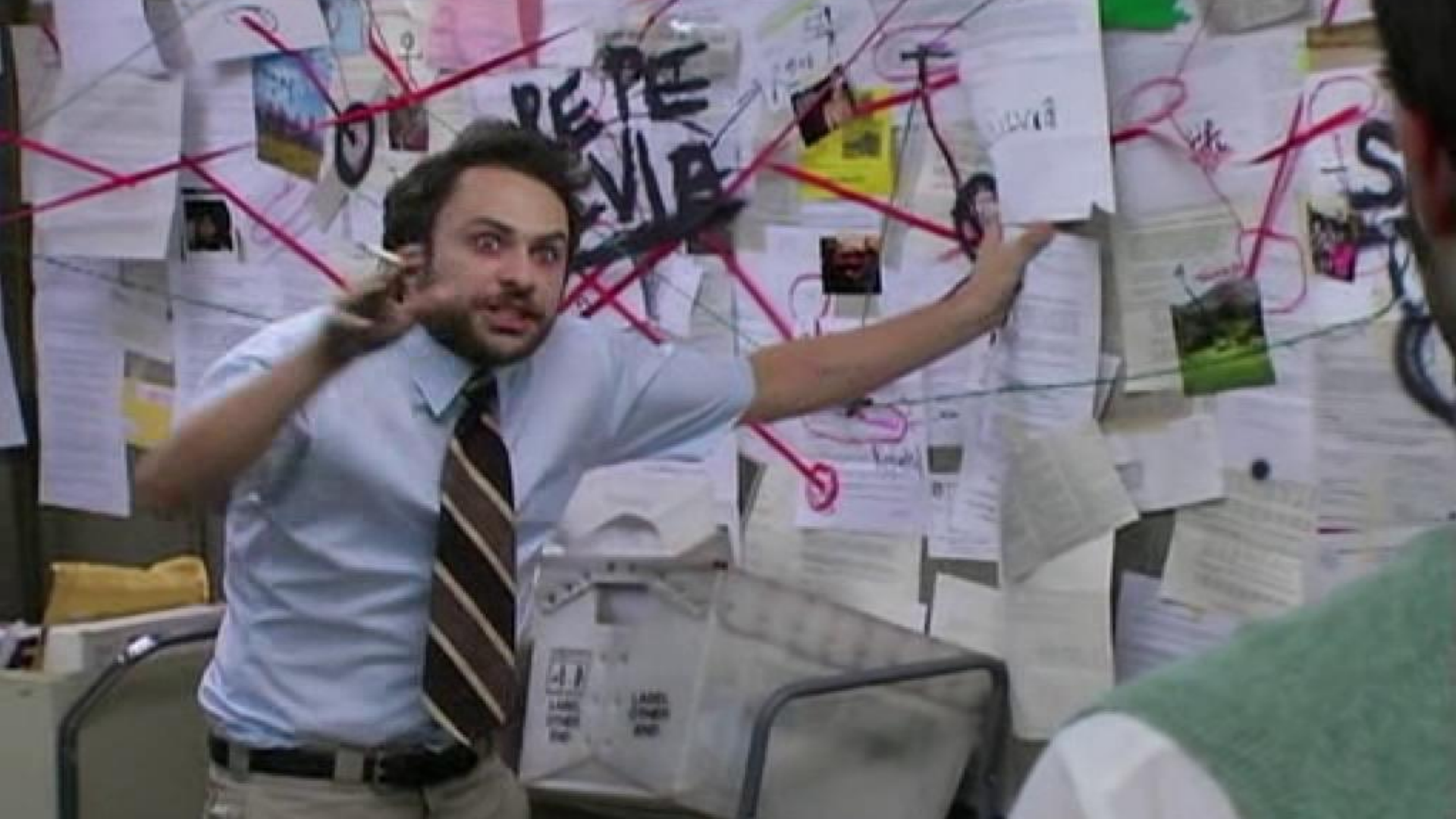
AMD Sinkclose

Universal SMM Privilege Escalation

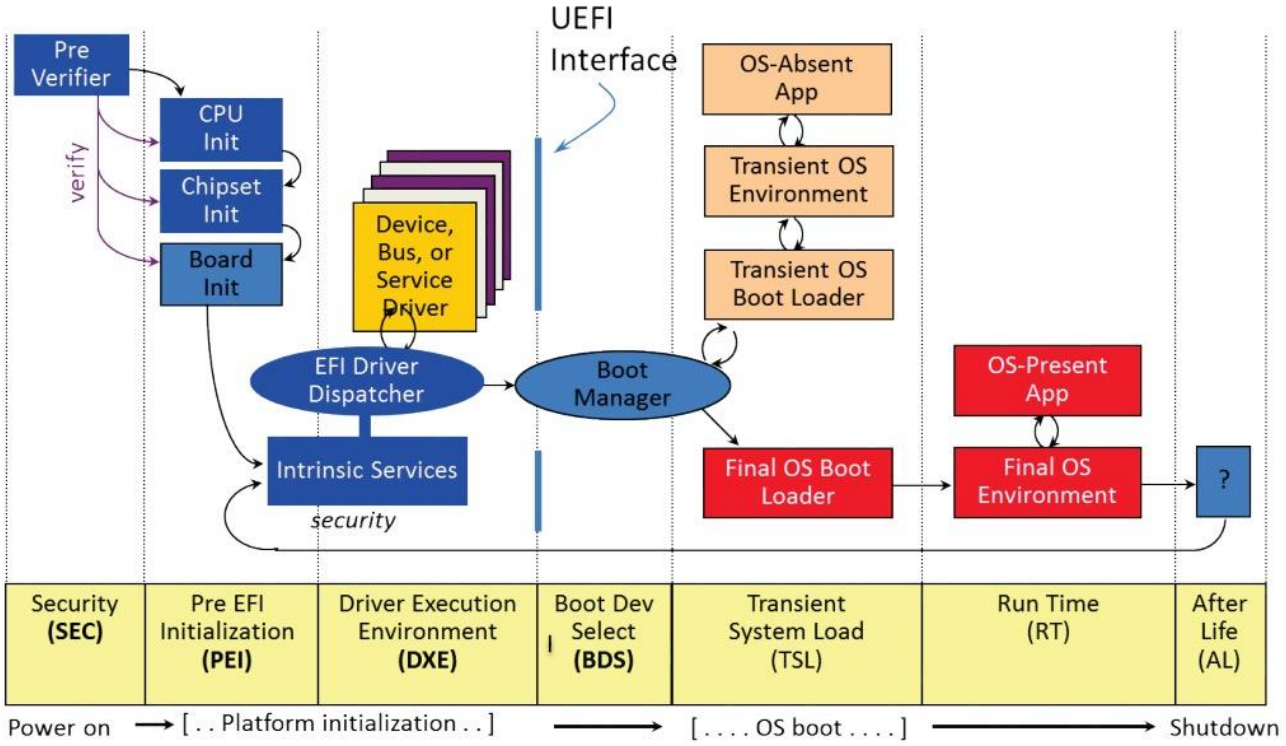
Enrique Nissim
Krzysztof Okupski

IOActive®



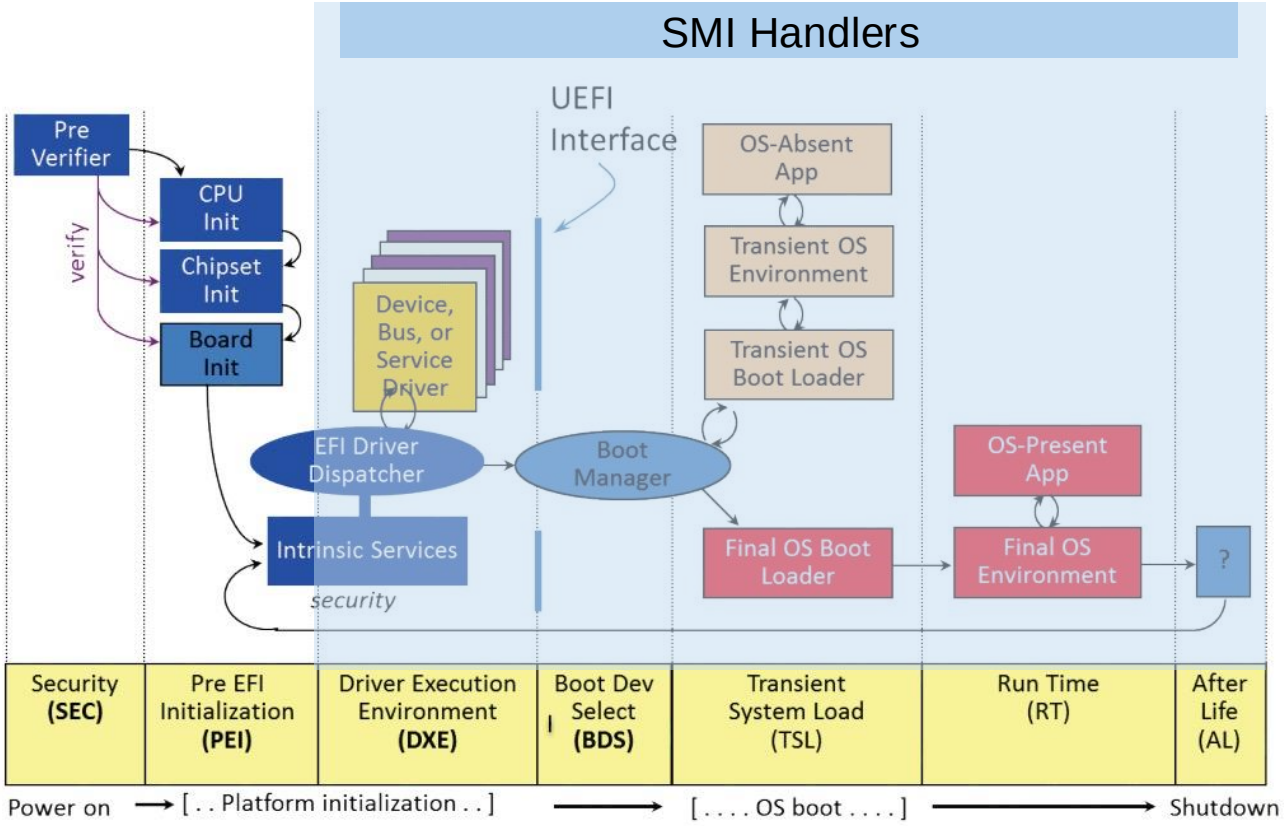


UEFI Boot Phases

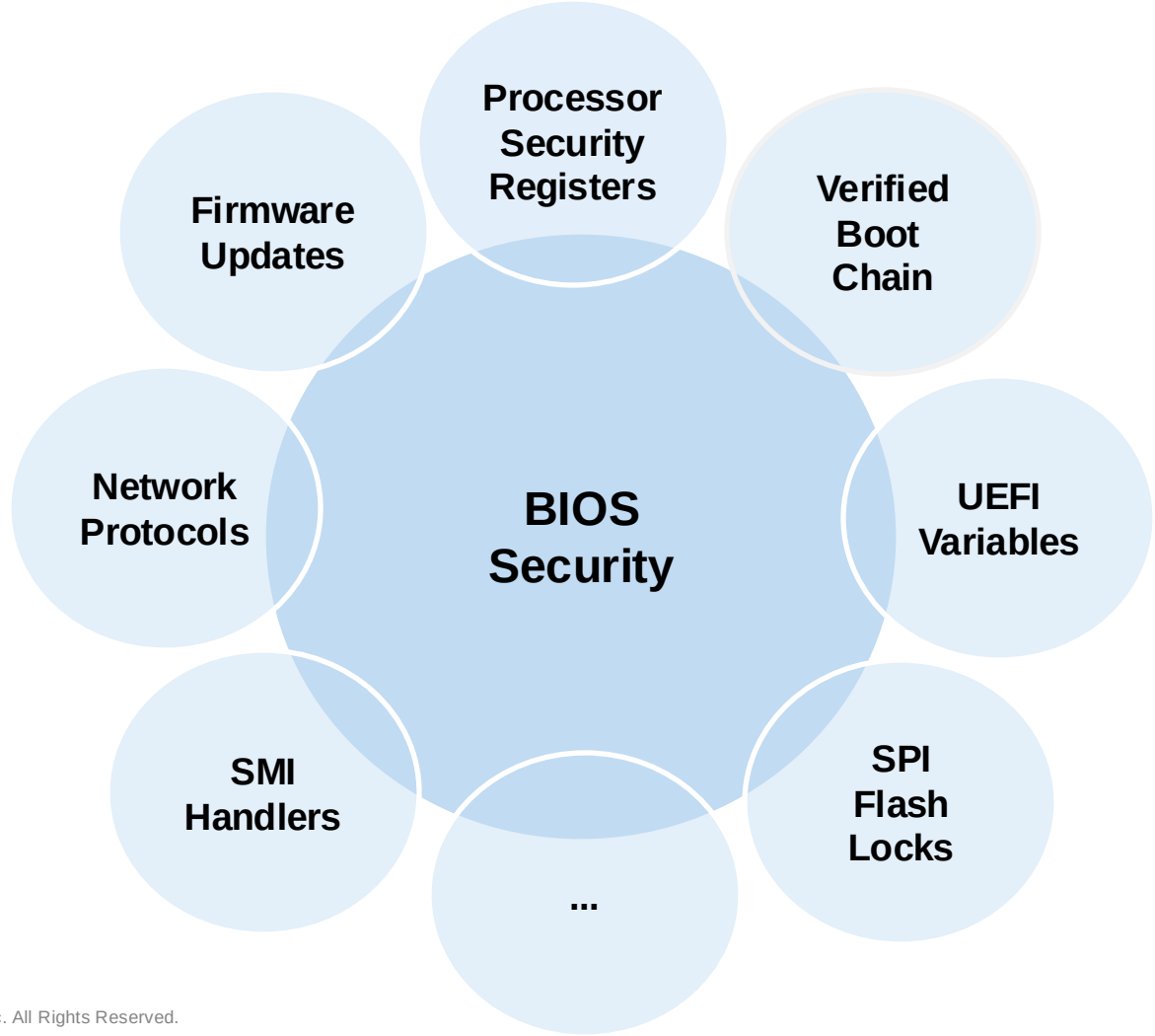


IOActive.

UEFI Boot Phases



IOActive.



IOActive.

IOActive research on AMD platform security

- Exploring AMD Platform Secure Boot (2023)
- Back to the Future with Platform Security (2023)
- Adventures in the Platform Security Coordinated Disclosure Circus (2023)
- Exploring the security configuration of AMD platforms (2022)

CVE-2023-20576	*CVE-2023-20577*	*CVE-2023-20579*
CVE-2023-20587	*CVE-2023-20596*	*CVE-2023-31100*
CVE-2023-28468	*CVE-2023-2290*	*CVE-2023-5078*

- https://github.com/IOActive/Platbox

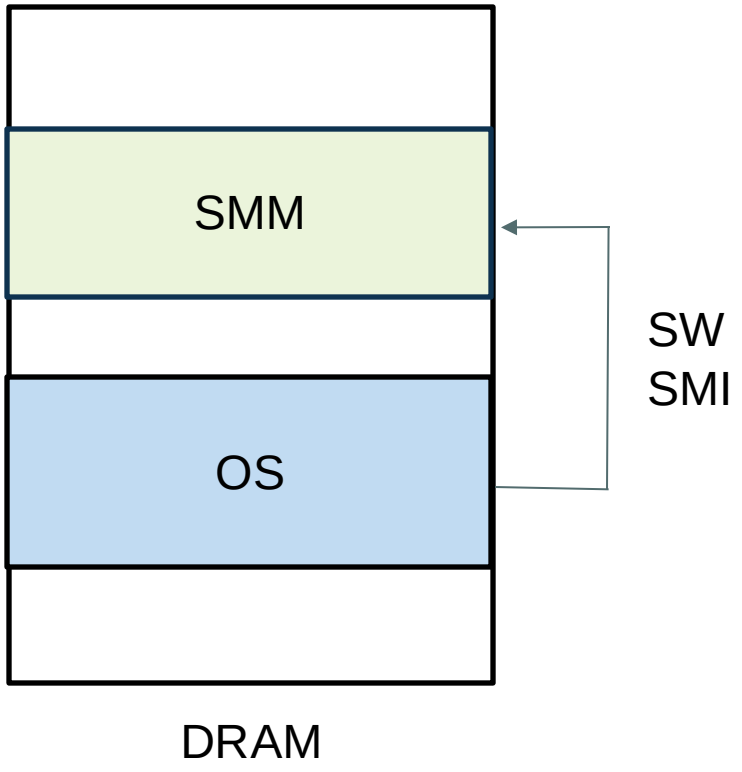
IOActive

About System Management Mode (1/2)

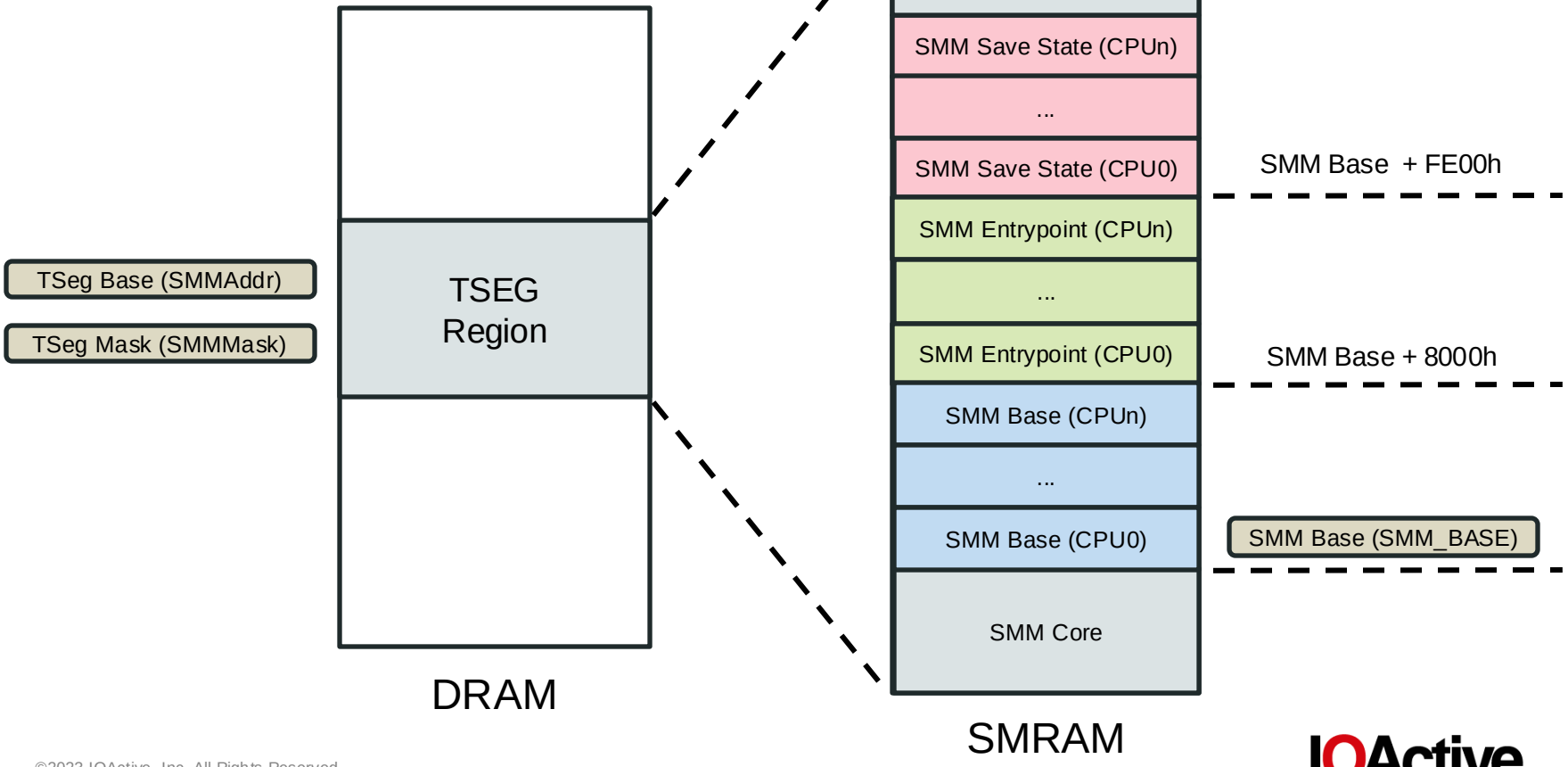
- One of the most powerful execution modes in x86
- Invisible to the OS and Hypervisor
- Full system memory and I/O devices access
- SMM can write to the SPI Flash
  - ROM protection registers can be disabled in AMD (potential for persistence)

About System Management Mode (2/2)

- SMM is entered using a special external interrupt called the system-management interrupt (SMI)
- After an SMI is received by the processor, the processor saves the processor state in a separate address space, called SMRAM



IOActive.



IOActive.

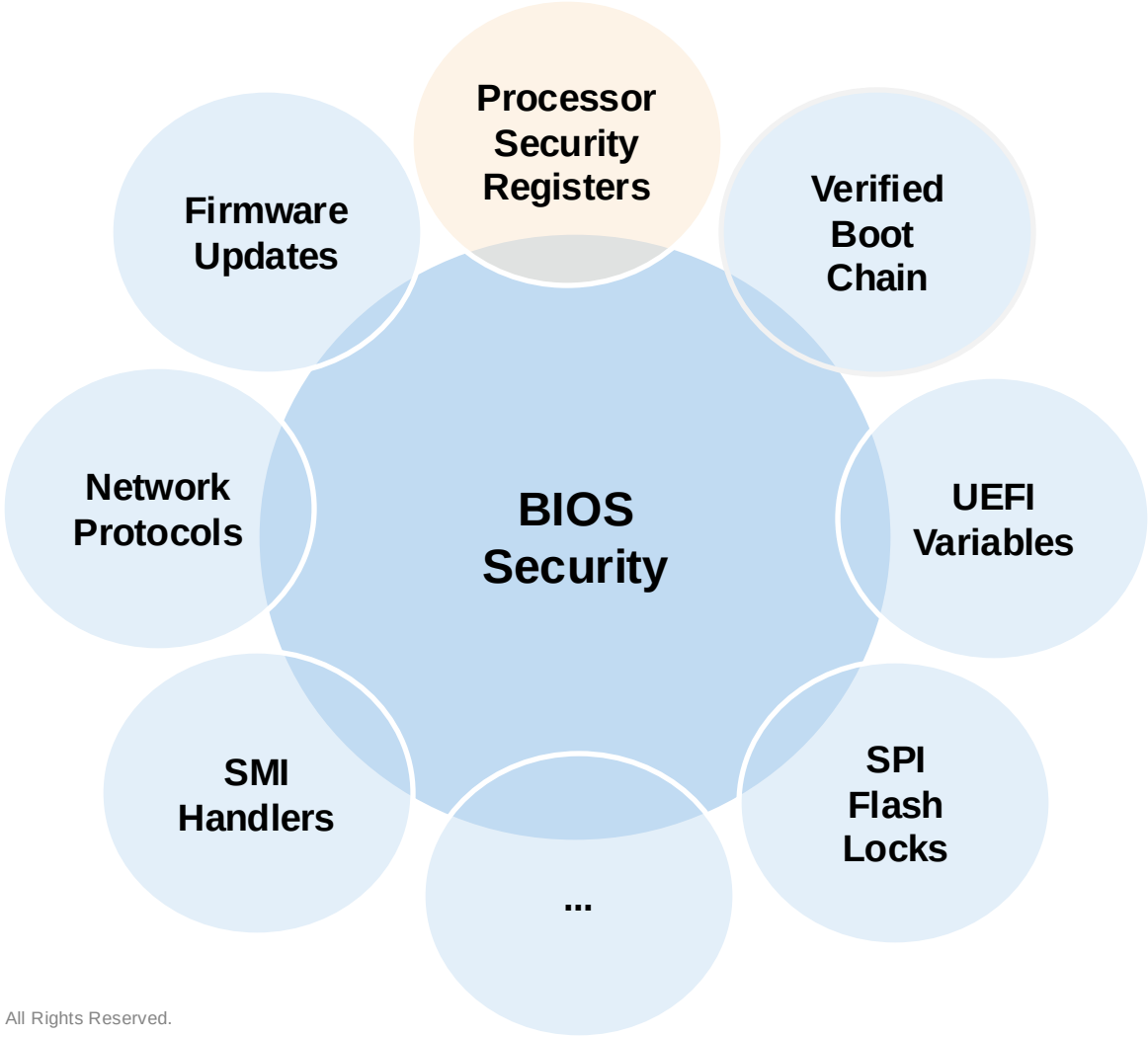
SMRAM Protection

- MSRC001_0111 (SMM_BASE used for SMM base address)
- MSRC001_0112 (SMM TSeg Base Address (SMMAddr))
- MSRC001_0113 (SMM TSeg Mask (SMMMask))
- MSRC001_0015[SmmLock] (HWCR used for locking the config)

These need to be configured for each core

AMD MSRs and behavioral difference with Intel

- On Intel systems, there are specific MSRs that are only accessible while the processor is executing at SMM
  - *example: IA32_SMBASE (base of SMM)*
- On AMD, there are no such restrictions. All the MSRs that make for the security of SMM are accessible from Ring 0.
  - *Note that when SmmLock is set, accesibility does not imply the configuration can be changed even from SMM*



IOActive.

Spotting the bug through the processor documentation

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

Differences with the "Memory Sinkhole"

- The Memory Sinkhole was limited to the APIC only
- The AMD Sinkclose permits any device to be remapped over TSEG... right?

Attack Idea

- Find a device with a BAR having register values such that when overlapped with the SMM Entry Point, we could take control of the execution

- We could also have our own custom PCIe device connected with a convinient BAR mapping

PCI BARs and Configuration Space

- There are multiple integrated devices in modern Ryzen processors
- We can try re-mapping the PCI Base Address Register from one of them to make it overlap with SMRAM
- The registers for the device should become visible for the OS at the TSEG location

PCI BARs failed

```
/dev/KernetixDriver0 opened successfully: 3
+ SMM region info:
TSEG Base    : bf000000
TSEG Size    : 00ffffff
SMM  Base    : bfea8000
SMM-Entry    : bfeb0000
Ethernet controller BAR2 at: d0714000
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
 -> remapping BAR2 to overlap TSEG
+ successfully overlaped the ethernet bar over SMM at: bfeb0000
 -> view of memory at smm entry point:
0xbfeb0000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

 -> Memory at BAR2 (d0714000):
0xd0714000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

Restoring BAR and dumping again:
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
```


IOActive.

PCI BARs failed

```
/dev/KernetixDriver0 opened successfully: 3
+ SMM region info:
TSEG Base    : bf000000
TSEG Size    : 00ffffff
SMM  Base    : bfea8000
SMM-Entry    : bfeb0000
Ethernet controller BAR2 at: d0714000
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
 -> remapping BAR2 to overlap TSEG
+ successfully overlaped the ethernet bar over SMM at: bfeb0000
 -> view of memory at smm entry point:
0xbfeb0000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

 -> Memory at BAR2 (d0714000):
0xd0714000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

Restoring BAR and dumping again:
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
```

Visible device registers

IOActive.

PCI BARs failed

```
/dev/KernetixDriver0 opened successfully: 3
+ SMM region info:
TSEG Base    : bf000000
TSEG Size    : 00ffffff
SMM  Base    : bfea8000
SMM-Entry    : bfeb0000
Ethernet controller BAR2 at: d0714000
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
 -> remapping BAR2 to overlap TSEG
+ successfully overlaped the ethernet bar over SMM at: bfeb0000
 -> view of memory at smm entry point:
0xbfeb0000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

 -> Memory at BAR2 (d0714000):
0xd0714000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

Restoring BAR and dumping again:
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
```

Visible device registers

Remap failed. Registers are not available

IOActive.

PCI BARs failed

```
/dev/KernetixDriver0 opened successfully: 3
+ SMM region info:
TSEG Base    : bf000000
TSEG Size    : 00ffffff
SMM  Base    : bfea8000
SMM-Entry    : bfeb0000
Ethernet controller BAR2 at: d0714000
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
 -> remapping BAR2 to overlap TSEG
+ successfully overlaped the ethernet bar over SMM at: bfeb0000
 -> view of memory at smm entry point:
0xbfeb0000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

 -> Memory at BAR2 (d0714000):
0xd0714000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

Restoring BAR and dumping again:
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
```

Visible device registers

Remap failed. Registers are not available

The BAR was indeed moved from its original place

IOActive.

PCI BARs failed

```
/dev/KernetixDriver0 opened successfully: 3
+ SMM region info:
TSEG Base    : bf000000
TSEG Size    : 00ffffff
SMM  Base    : bfea8000
SMM-Entry    : bfeb0000
Ethernet controller BAR2 at: d0714000
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
 -> remapping BAR2 to overlap TSEG
+ successfully overlaped the ethernet bar over SMM at: bfeb0000
 -> view of memory at smm entry point:
0xbfeb0000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xbfeb0020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

 -> Memory at BAR2 (d0714000):
0xd0714000 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714010 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xd0714020 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................

Restoring BAR and dumping again:
0xd0714000 | 00 2b 67 52 7c c0 00 00 40 00 00 00 80 00 00 00 | .+gR|...@.......
0xd0714010 | 00 c0 ff ff 00 00 00 00 08 07 06 00 00 00 00 00 | ................
0xd0714020 | 00 b0 ff ff 00 00 00 00 00 00 00 00 00 00 00 00 | ................
```

Visible device registers

Remap failed. Registers are not available

The BAR was indeed moved from its original place

After restoration

IOActive.

TOM - Top of Memory

MSRC001_001A Top Of Memory (TOP_MEM)

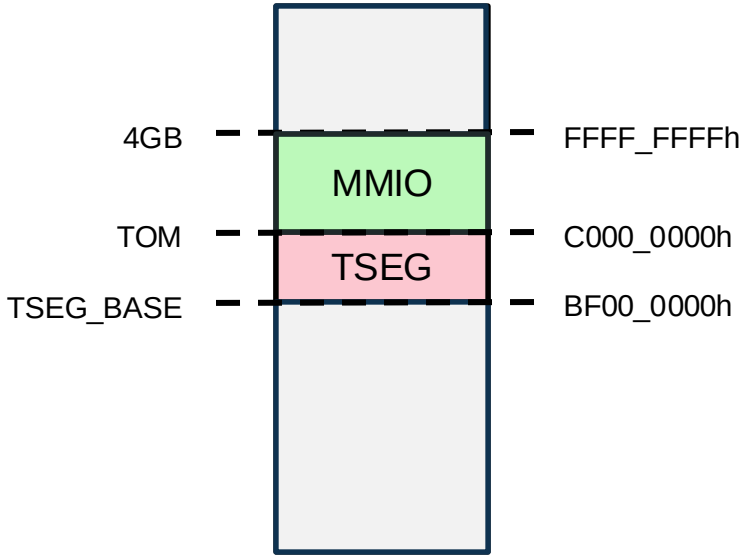
Reset: 0000_0000_0000_0000h.

Bits	Description
63:40	RAZ.
39:23	**TOM[39:23]: top of memory**. Read-write. Specifies the address that divides between MMIO and DRAM. This value is normally placed below 4G. From TOM to 4G is MMIO; below TOM is DRAM. See 2.4.6 [System Address Map] and 2.9.11 [DRAM CC6/PC6 Storage].
22:0	RAZ.

- This register dictates where the MMIO region below 4G starts
- In the Intel platform this register has a lock bit and cannot be modified when set
- There is no such lock in AMD :)

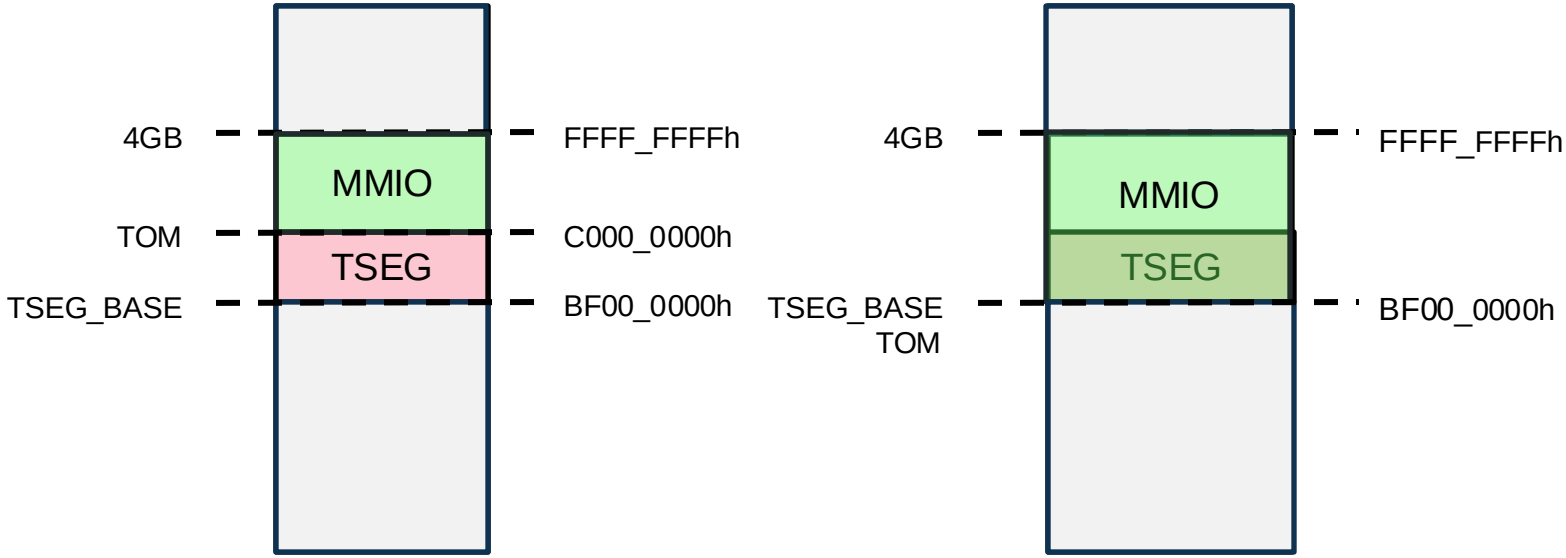
IOActive.

Moving TOM down



IOActive.

Moving TOM down



IOActive.

EMBARGOED

IOActive.

Memory routing priorities

2.4.6.1.2 Determining The Access Destination for Core Accesses

The access destination, DRAM or MMIO, is based on the highest priority of the following ranges that the access falls in: 1==Lowest priority.

1. RdDram/WrDram as determined by MSRC001_001A [Top Of Memory (TOP_MEM)] and MSRC001_001D [Top Of Memory 2 (TOM2)].
2. The IORRs. (see MSRC001_00[18,16] and MSRC001_00[19,17]).
3. The fixed MTRRs. (see MSR0000_02[6F:68,59:58,50] [Fixed-Size MTRRs])
4. TSeg & ASeg SMM mechanism. (see MSRC001_0112 and MSRC001_0113)
5. MMIO config space, APIC space.
   - MMIO APIC space and MMIO config space must not overlap.
   - RdDram=IO, WrDram=IO.
   - See 2.4.9.1.2 [APIC Register Space] and 2.7 [Configuration Space].
6. NB address space routing. See 2.8.2.1.1 [DRAM and MMIO Memory Space].

Memory routing priorities

2.4.6.1.2 Determining The Access Destination for Core Accesses

The access destination, DRAM or MMIO, is based on the highest priority of the following ranges that the access falls in: 1==Lowest priority.

1. RdDram/WrDram as determined by MSRC001_001A [Top Of Memory (TOP_MEM)] and MSRC001_001D [Top Of Memory 2 (TOM2)].
2. The IORRs. (see MSRC001_00[18,16] and MSRC001_00[19,17]).
3. The fixed MTRRs. (see MSR0000_02[6F:68,59:58,50] [Fixed-Size MTRRs])
4. TSeg & ASeg SMM mechanism. (see MSRC001_0112 and MSRC001_0113)
5. MMIO config space, APIC space.
   - MMIO APIC space and MMIO config space must not overlap.
   - RdDram=IO, WrDram=IO.
   - See 2.4.9.1.2 [APIC Register Space] and 2.7 [Configuration Space].
6. NB address space routing. See 2.8.2.1.1 [DRAM and MMIO Memory Space].

IOActive.

Memory routing priorities

2.4.6.1.2 Determining The Access Destination for Core Accesses

The access destination, DRAM or MMIO, is based on the highest priority of the following ranges that the access falls in: 1==Lowest priority.

1. RdDram/WrDram as determined by MSRC001_001A [Top Of Memory (TOP_MEM)] and MSRC001_001D [Top Of Memory 2 (TOM2)].
2. The IORRs. (see MSRC001_00[18,16] and MSRC001_00[19,17]).
3. The fixed MTRRs. (see MSR0000_02[6F:68,59:58,50] [Fixed-Size MTRRs])
4. TSeg & ASeg SMM mechanism. (see MSRC001_0112 and MSRC001_0113)
5. MMIO config space, APIC space.
   - MMIO APIC space and MMIO config space must not overlap.
   - RdDram=IO, WrDram=IO.
   - See 2.4.9.1.2 [APIC Register Space] and 2.7 [Configuration Space].
6. NB address space routing. See 2.8.2.1.1 [DRAM and MMIO Memory Space].

IOActive.

Memory routing priorities

2.4.6.1.2 Determining The Access Destination for Core Accesses

The access destination, DRAM or MMIO, is based on the highest priority of the following ranges that the access falls in: 1==Lowest priority.

1. RdDram/WrDram as determined by MSRC001_001A [Top Of Memory (TOP_MEM)] and MSRC001_001D [Top Of Memory 2 (TOM2)].
2. The IORRs. (see MSRC001_00[18,16] and MSRC001_00[19,17]).
3. The fixed MTRRs. (see MSR0000_02[6F:68,59:58,50] [Fixed-Size MTRRs])
4. TSeg & ASeg SMM mechanism. (see MSRC001_0112 and MSRC001_0113)
5. MMIO config space, APIC space.
   - MMIO APIC space and MMIO config space must not overlap.
   - RdDram=IO, WrDram=IO.
   - See 2.4.9.1.2 [APIC Register Space] and 2.7 [Configuration Space].
6. NB address space routing. See 2.8.2.1.1 [DRAM and MMIO Memory Space]. ?

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0   ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```


IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0     ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```


IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0     ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0     ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

writes size into GDT descriptor below

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0     ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

writes size into GDT descriptor below

loads GDT base

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0     ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

writes size into GDT descriptor below

loads GDT base

writes GDT base into the descriptor

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0   ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

writes size into GDT descriptor below

loads GDT base

writes GDT base into the descriptor

Loads the GDTR with the descriptor

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0   ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

writes size into GDT descriptor below

loads GDT base

writes GDT base into the descriptor

Loads the GDTR with the descriptor

preapres the CS for the far jmp

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0   ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

writes size into GDT descriptor below

loads GDT base

writes GDT base into the descriptor

Loads the GDTR with the descriptor

preapres the CS for the far jmp

prepares the offset (target) of the far jmp

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0   ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

writes size into GDT descriptor below

loads GDT base

writes GDT base into the descriptor

Loads the GDTR with the descriptor

preapres the CS for the far jmp

prepares the offset (target) of the far jmp

enables protected mode

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0   ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

writes size into GDT descriptor below

loads GDT base

writes GDT base into the descriptor

Loads the GDTR with the descriptor

preapres the CS for the far jmp

prepares the offset (target) of the far jmp

enables protected mode

jumps to 32-bit code (protected mode)

IOActive.

Analysis of the EDKII SMM Entry Point

```
BITS 16
ASM_PFX(gcSmiHandlerTemplate):
_SmiEntryPoint:
    mov     bx, _GdtDesc - _SmiEntryPoint + 0x8000
    mov     ax,[cs:DSC_OFFSET + DSC_GDTSIZ]
    dec     ax
    mov     [cs:bx], ax
    mov     eax, [cs:DSC_OFFSET + DSC_GDTPTR]
    mov     [cs:bx + 2], eax
o32 lgdt    [cs:bx]                 ; lgdt fword ptr cs:[bx]
    mov     ax, PROTECT_MODE_CS
    mov     [cs:bx-0x2],ax
    mov     edi, strict dword 0   ; source operand will be patched
ASM_PFX(gPatchSmbase):
    lea     eax, [edi + (@ProtectedMode - _SmiEntryPoint) + 0x8000]
    mov     [cs:bx-0x6],eax
    mov     ebx, cr0
    and     ebx, 0x9ffafff3
    or      ebx, 0x23
    mov     cr0, ebx
    jmp     dword 0x0:0x0
_GdtDesc:
    DW 0
    DD 0

BITS 32
@ProtectedMode:
...
```

loads GDT size

writes size into GDT descriptor below

loads GDT base

writes GDT base into the descriptor

Loads the GDTR with the descriptor

preapres the CS for the far jmp

prepares the offset (target) of the far jmp

enables protected mode

jumps to 32-bit code (protected mode)

IOActive.

Analysis of the EDKII SMM Entry Point

```
 0:   bb 4d 80                mov    bx,0x804d     ; 0x8000 + 0x4D
 3:   2e a1 d8 fd             mov    ax,cs:0xfdd8  ; DSC_OFFSET + 0xD8
 7:   48                      dec    ax
 8:   2e 89 07                mov    WORD PTR cs:[bx],ax
 b:   2e 66 a1 d0 fd          mov    eax,cs:0xfdd0 ; DSC_OFFSET + 0xD0
10:   2e 66 89 47 02          mov    DWORD PTR cs:[bx+0x2],eax
15:   2e 66 0f 01 17          lgdtd  cs:[bx];
1a:   b8 08 00                mov    ax,0x8
1d:   2e 89 47 fe             mov    WORD PTR cs:[bx-0x2],ax
21:   66 bf 00 30 f4 ae       mov    edi,0xaef43000
27:   66 67 8d 87 53 80 00    lea    eax,[edi+0x8053]
2e:   00
2f:   2e 66 89 47 fa          mov    DWORD PTR cs:[bx-0x6],eax
34:   0f 20 c3                mov    ebx,cr0
37:   66 81 e3 f3 ff fa 9f    and    ebx,0x9ffafff3
3e:   66 83 cb 23             or     ebx,0x23
42:   0f 22 c3                mov    cr0,ebx
45:   66 ea 53 b0 f4 ae 08    jmp    0x8:0xaef4b053
4c:   00
4d:   [ GDTR HERE ]
```


IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

APIC Registers

```
>>> rdmsr 0x1b
-> MSR:[0000001b]: fee00800
>>> physmem r 0xfee00000 0x100
0xfee00000 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
0xfee00010 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
0xfee00020 | 00 00 00 06 00 00 00 06 00 00 00 06 00 00 00 06 | ................
0xfee00030 | 10 00 05 80 10 00 05 80 10 00 05 80 10 00 05 80 | ................
0xfee00040 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
0xfee00050 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
0xfee00060 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
0xfee00070 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
0xfee00080 | 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00 00 | ................
0xfee00090 | 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00 00 | ................
0xfee000a0 | 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00 00 | ................
0xfee000b0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
0xfee000c0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
0xfee000d0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
0xfee000e0 | ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff | ................
0xfee000f0 | ff 01 00 00 ff 01 00 00 ff 01 00 00 ff 01 00 00 | ................
```

Reserved region

Writes are discarded

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

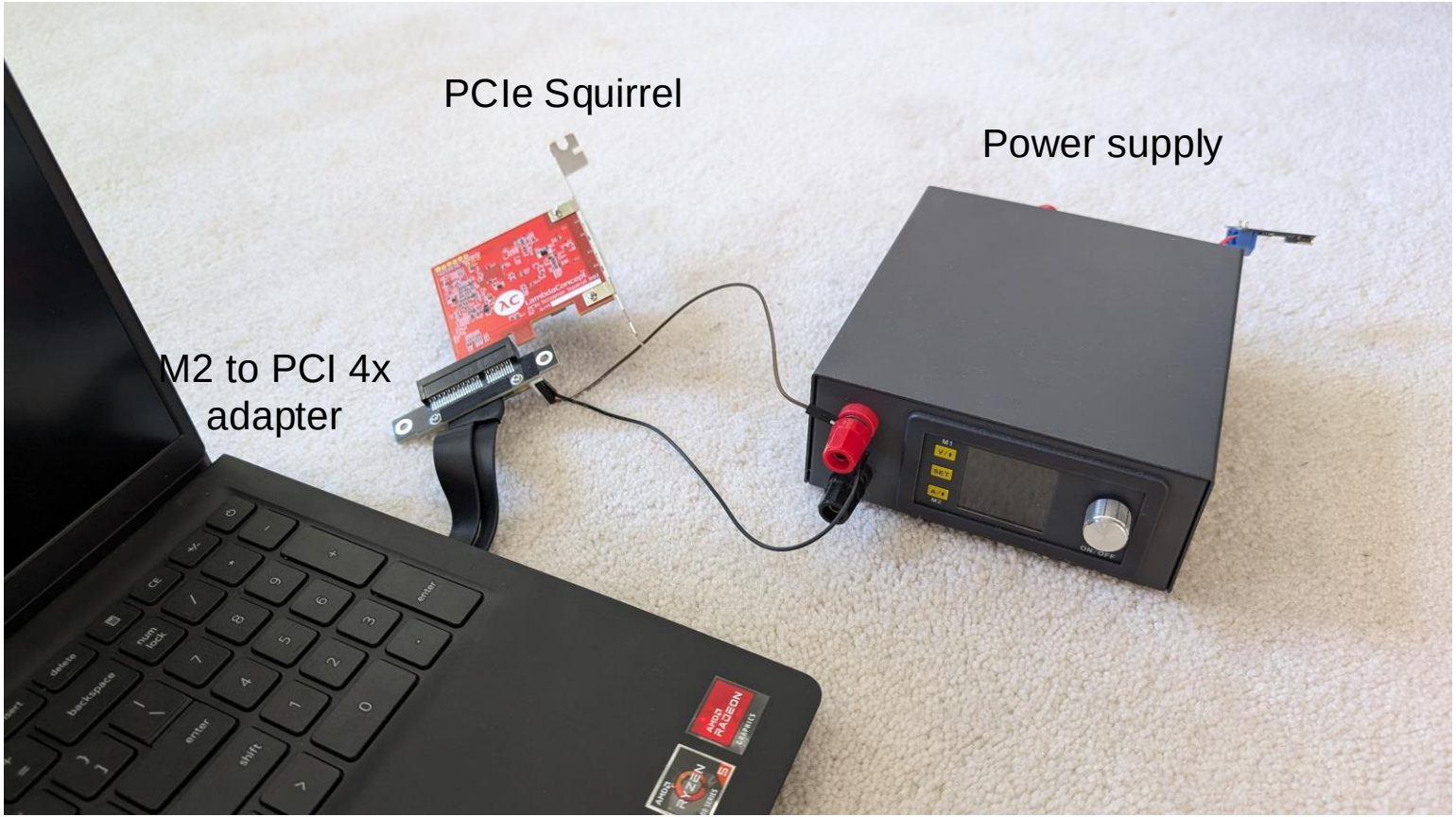
IOActive.

EMBARGOED

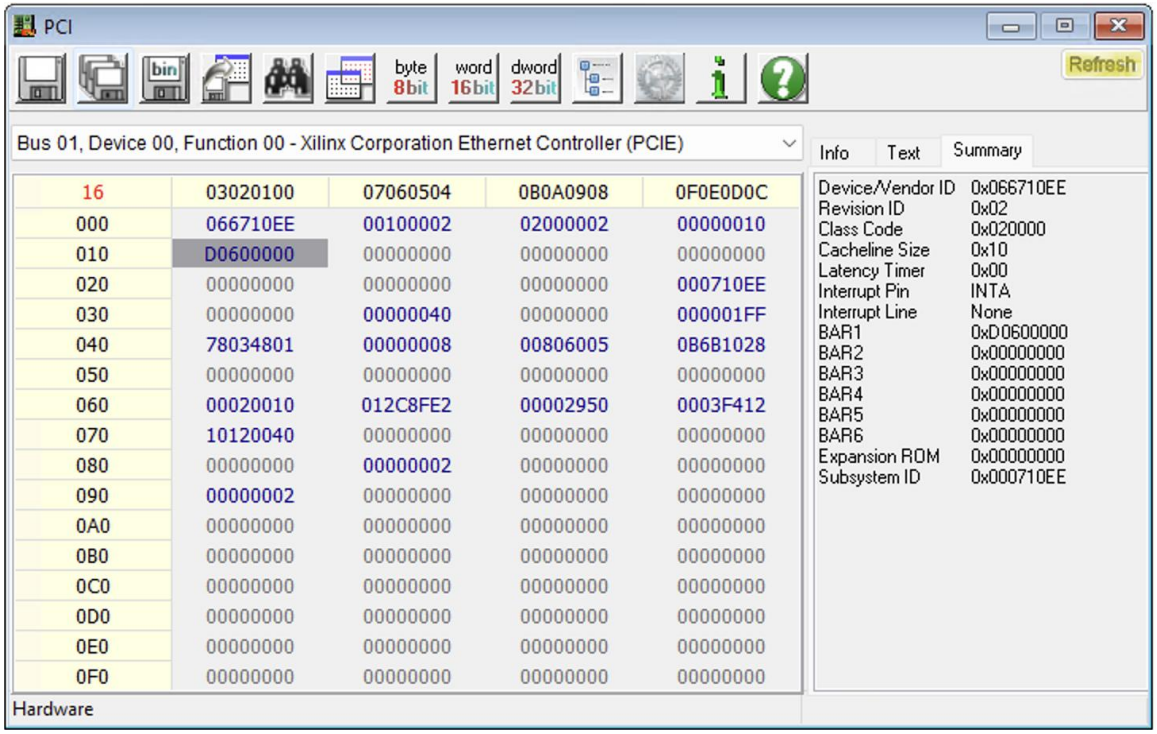
IOActive.

Debugging Setup

- BAR buffer
  - PCI Squirrel with PCILeech firmware
  - Enabled via command register (bit 1) in device BAR
- SMM backdoor
  - Used for modifying code in SMM on-demand



IOActive.



IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

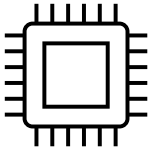
EMBARGOED

IOActive.

EMBARGOED

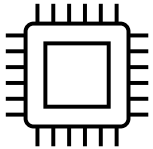
IOActive.

SMIs explained



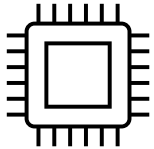
Ring 0

```
xor eax, eax
xor eax, eax
```



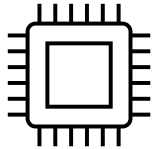
Ring 0

```
xor eax, eax
xor eax, eax
```



Ring 0

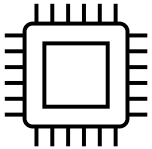
```
xor eax, eax
xor eax, eax
```



Ring 0

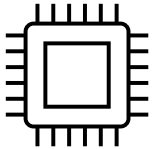
```
xor eax, eax
xor eax, eax
```

SMIs explained



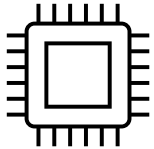
Ring 0

```
xor eax, eax
xor eax, eax
smi
```



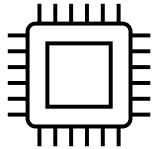
Ring 0

```
xor eax, eax
xor eax, eax
```



Ring 0

```
xor eax, eax
xor eax, eax
```

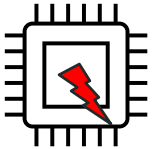


Ring 0

```
xor eax, eax
xor eax, eax
```

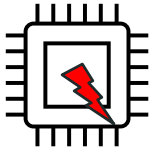

IOActive.

SMIs explained



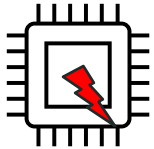
Ring 0

```
xor eax, eax
xor eax, eax
smi
```



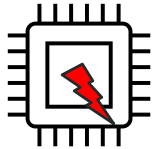
Ring 0

```
xor eax, eax
xor eax, eax
```



Ring 0

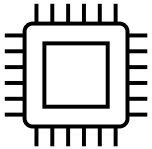
```
xor eax, eax
xor eax, eax
```



Ring 0

```
xor eax, eax
xor eax, eax
```

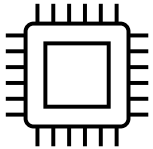
SMIs explained



Ring -2

```
xor eax, eax
xor eax, eax
smi
```

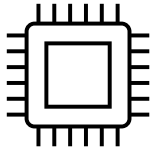
```
mov bs, 0x804d
mov ax, cs:0xfdd8
...
```



Ring -2

```
xor eax, eax
xor eax, eax
```

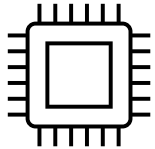
```
mov bs, 0x804d
mov ax, cs:0xfdd8
...
```



Ring -2

```
xor eax, eax
xor eax, eax
```

```
mov bs, 0x804d
mov ax, cs:0xfdd8
...
```



Ring -2

```
xor eax, eax
xor eax, eax
```

```
mov bs, 0x804d
mov ax, cs:0xfdd8
...
```


IOActive.

SMIs explained

- Initially thought that SMIs are local, but they are global

- Triggering an SMI on any core will:
  - Assert an interrupt on all cores
  - Make the switch after the current instruction finishes
  - Store the current register state at SMM base + 0xFE00
  - Execute the entry point at SMM base + 0x8000

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

DEMO

EMBARGOED

IOActive.

Platform security

Vendor	Model	PSB State	ROM Armor State
Acer	Swift 3 SF314-42	Not configured	Not configured
Acer	TravelMate P414-41	Not configured	Not configured
ASUS	Strix G513QR	Not configured	Not configured
Lenovo	Thinkpad P16s	Configured*	Not configured
Lenovo	IdeaPad 1	Not configured	Not configured
Lenovo	Thinkpad T495s	Not configured	Not configured
Huawei	Matebook D16	Not configured	Not configured
HP	15s	Not configured	Not configured
Microsoft	Surface 4	Configured	Unknown
MSI	Bravo 15	Not configured	Not configured

IOActive.

EMBARGOED

IOActive.

EMBARGOED

IOActive.

Questions?

IOActive.